

Project 2 – VPC Peering with Public & Private Subnets

1. Project Objective & Architecture Plan

The objective of this project is to build a multi-VPC cloud infrastructure containing isolated public and private subnets, and establish a secure **VPC Peering Connection** between them. This network architecture allows resources in completely separate networks to communicate securely using private IP routing without exposing data to the public internet.

Network Architecture Configuration

- **VPC-A Network Range:** 10.0.0.0/16
 - **Public Subnet:** 10.0.1.0/24 (Availability Zone: ap-south-1a)
 - **Private Subnet:** 10.0.2.0/24 (Availability Zone: ap-south-1a)
 - **VPC-B Network Range:** 10.1.0.0/16
 - **Public Subnet:** 10.1.1.0/24
 - **Private Subnet:** 10.1.2.0/24
-

2. Step-by-Step Implementation Procedure

Step 2.1: Provisioning and Configuring VPC-A

1. Navigate to the **AWS VPC Console**, click **Create VPC**, name the resource VPC-A, and assign the CIDR block 10.0.0.0/16.
2. Create two distinct subnets within VPC-A: VPC-A-Public (10.0.1.0/24) and VPC-A-Private (10.0.2.0/24).
3. Create a new **Internet Gateway (IGW)**, name it VPC-A-IGW, and attach it directly to VPC-A.
4. Open the public route table associated with VPC-A and add a route directing all outbound internet traffic (0.0.0.0/0) to the VPC-A-IGW.
5. Associate this public route table with the VPC-A-Public subnet and enable **Auto-assign public IPv4 address**.

aws | vpcs | VPC | ap south 1

ap south 1.console.aws.amazon.com/vpcconsole/home?region=ap-south-1#vpcs:

VPC > Your VPCs

VPC dashboard <

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

DHCP option sets

Elastic IPs

Managed prefix lists

NAT gateways

Peering connections

Route servers

Security

Network ACLs

Security groups

PrivateLink and Lattice

Your VPCs

VPCs VPC encryption controls

Your VPCs (1) Info

Find VPCs by attribute or tag

☐	Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public ..
☐	VPC-A	vpc-0c15a6b46dba96671	Available	-	-	Off

Select a VPC above

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

aws | subnets | VPC | ap south 1

ap south 1.console.aws.amazon.com/vpcconsole/home?region=ap-south-1#subnets:

VPC > Subnets

VPC dashboard <

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

DHCP option sets

Elastic IPs

Managed prefix lists

NAT gateways

Peering connections

Route servers

Security

Network ACLs

Security groups

PrivateLink and Lattice

Subnets (2) Info

Find subnets by attribute or tag

☐	Name	Subnet ID	State	VPC	Block Public ...	IP
☐	VPC-A-public	subnet-049def6c955a1eb0b	Available	vpc-0c15a6b46dba96671 VPC-A	Off	10
☐	VPC-A-private	subnet-0a4ce4500405f9163	Available	vpc-0c15a6b46dba96671 VPC-A	Off	10

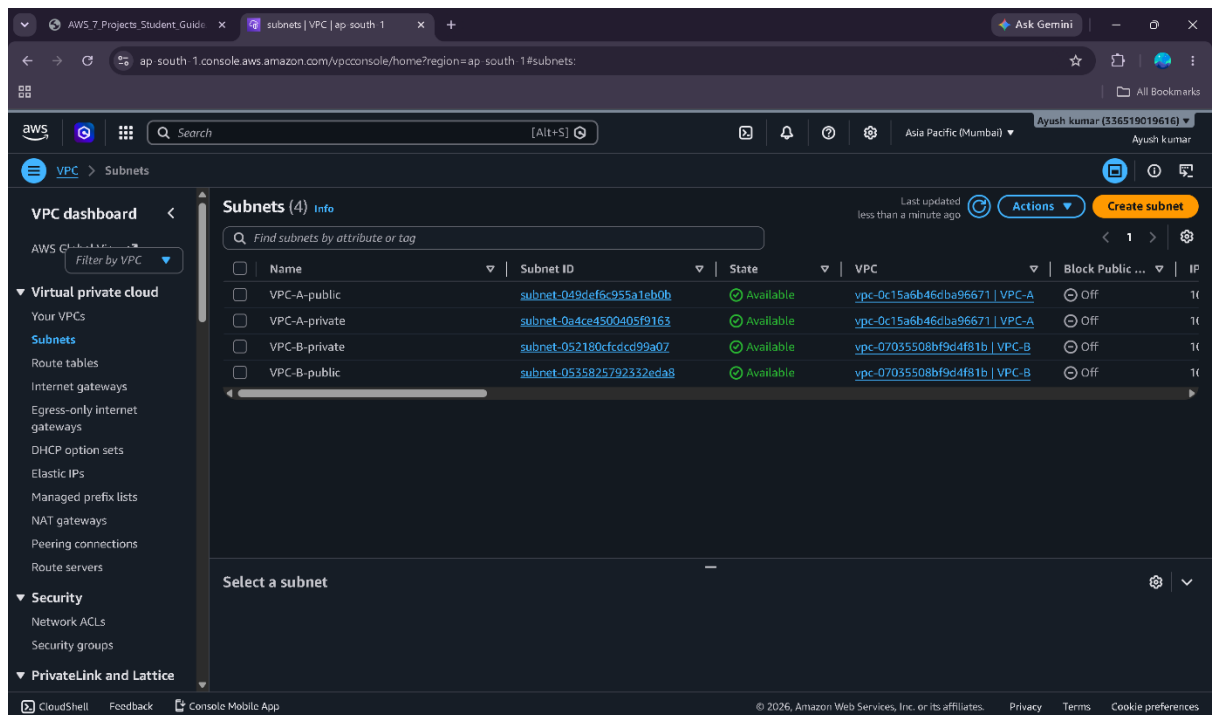
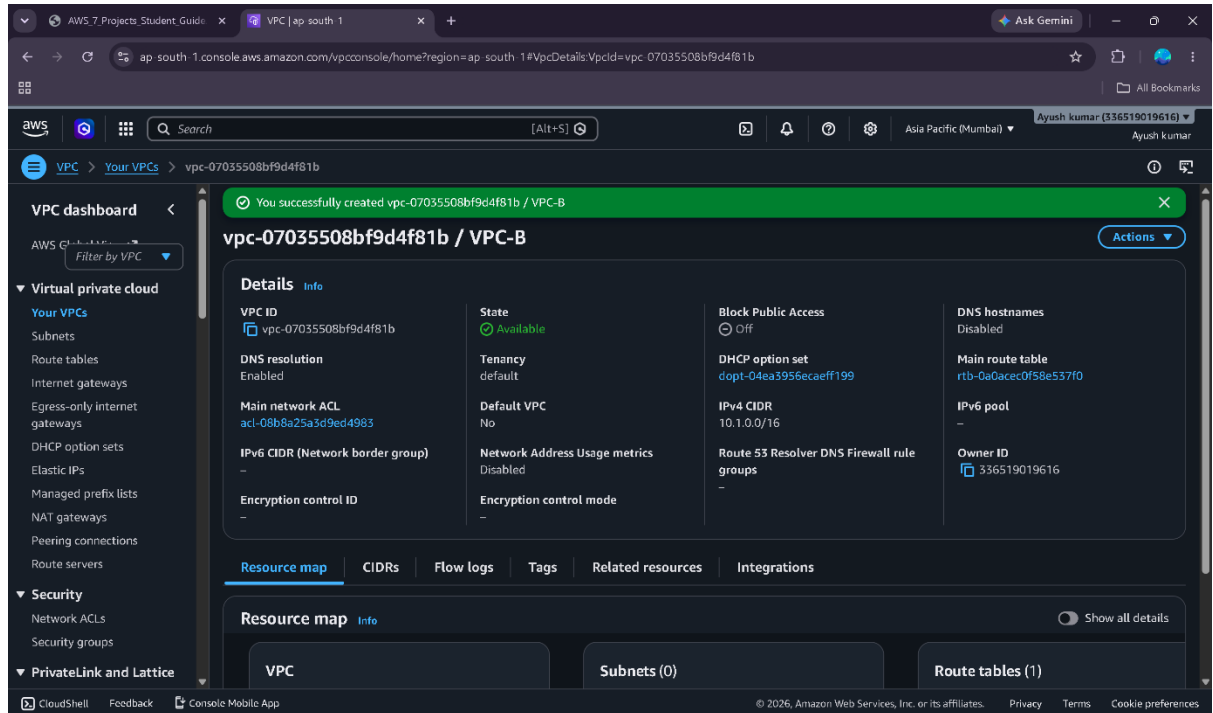
Select a subnet

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

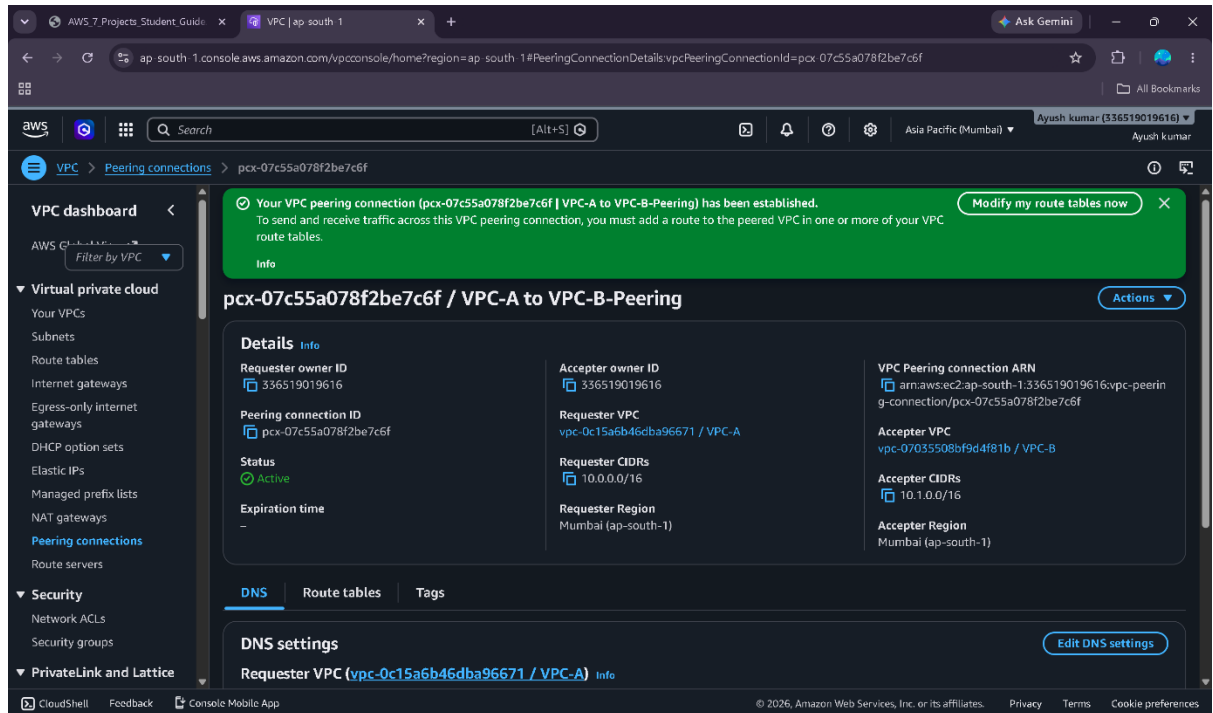
Step 2.2: Provisioning and Configuring VPC-B

1. In the VPC Console, create the second isolated network named VPC-B using the CIDR block 10.1.0.0/16.
2. Create two subnets inside VPC-B: VPC-B-Public (10.1.1.0/24) and VPC-B-Private (10.1.2.0/24).



Step 2.3: Establishing the VPC Peering Connection

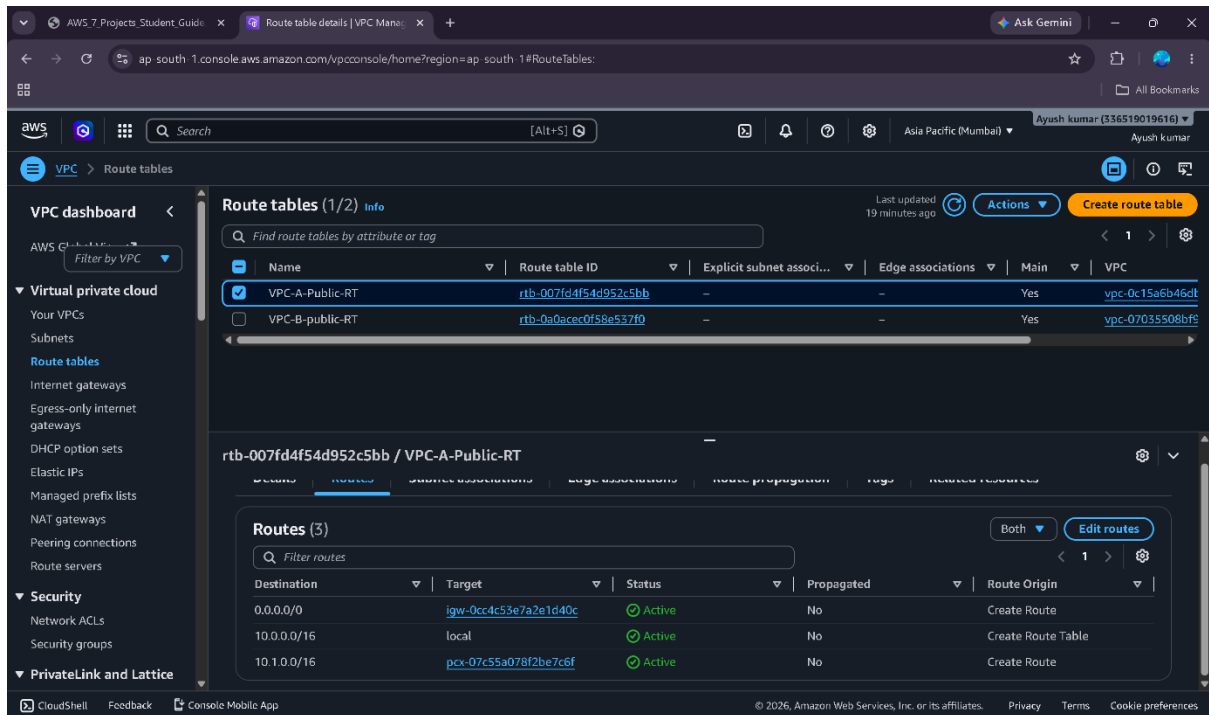
1. In the left navigation pane of the VPC console, select **Peering Connections** and click **Create peering connection**.
2. Configure the peering connection request by selecting VPC-A as the **Requester** network and VPC-B as the **Accepter** network.
3. Select the connection record from the dashboard, click **Actions**, and select **Accept Request** to activate the cross-VPC peering tunnel. This assigns a unique Peering Connection ID (pcx-xxxxx).



Step 2.4: Updating Route Tables for Cross-VPC Traffic

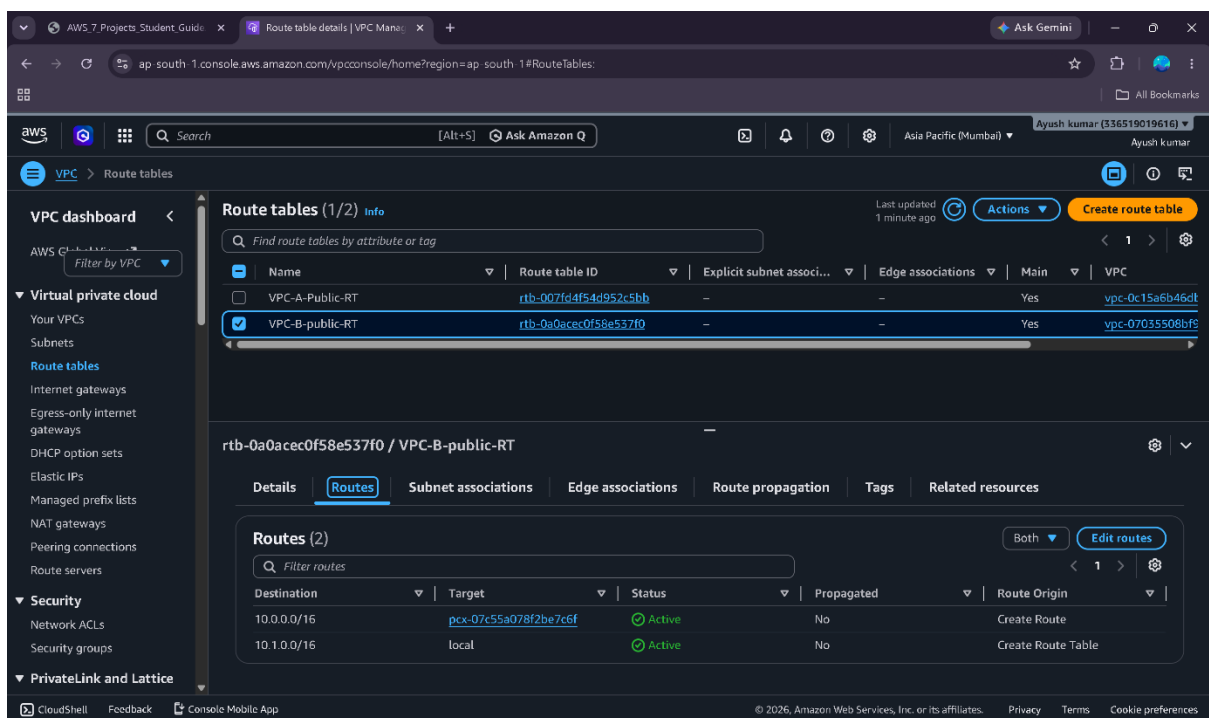
To ensure data packets are routed correctly over the new peering link, the respective routing tables must be updated manually:

- **VPC-A Route Table Update:** Add a network route where the **Destination** is 10.1.0.0/16 pointing to the **Target** peering ID pcx-xxxxx.
- **VPC-B Route Table Update:** Add a network route where the **Destination** is 10.0.0.0/16 pointing to the **Target** peering ID pcx-xxxxx.



The screenshot shows the AWS Management Console for the 'Route tables' section. The left sidebar shows the 'VPC dashboard' and 'Virtual private cloud' options. The main content area displays 'Route tables (1/2)' with a table listing two route tables: 'VPC-A-Public-RT' (rtb-007fd4f5d952c5bb) and 'VPC-B-public-RT' (rtb-0a0acec0f58e537f0). The 'VPC-A-Public-RT' is selected, and its details are shown below. The 'Routes (3)' section lists three routes:

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-0cc4c53e7a2e1d40c	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table
10.1.0.0/16	pcx-07c55a078f2be7c6f	Active	No	Create Route



The screenshot shows the AWS Management Console for the 'Route tables' section. The left sidebar shows the 'VPC dashboard' and 'Virtual private cloud' options. The main content area displays 'Route tables (1/2)' with a table listing two route tables: 'VPC-A-Public-RT' (rtb-007fd4f5d952c5bb) and 'VPC-B-public-RT' (rtb-0a0acec0f58e537f0). The 'VPC-B-public-RT' is selected, and its details are shown below. The 'Routes (2)' section lists two routes:

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	pcx-07c55a078f2be7c6f	Active	No	Create Route
10.1.0.0/16	local	Active	No	Create Route Table

Step 2.5: Security Group Rules Configuration

Modify the security groups within both networks to permit internal diagnostic traffic:

- Add an inbound rule allowing **ICMP (All traffic)** from the CIDR block of the opposing VPC.
- Add an inbound rule allowing **SSH (Port 22)** from the opposing VPC range to facilitate administrative management across the peering network.

The screenshot displays the AWS Management Console interface for the 'ap-south-1' region. The 'Security Groups' page is active, showing a list of security groups. The 'VPC-A SG' (sg-0b302c7d1d4482d2e) is selected, and its 'Inbound rules' tab is open. A green notification banner at the top indicates that inbound security group rules were successfully modified. The 'Inbound rules' table shows three rules: 'All traffic' (sgr-07d2e61acc343314a), 'SSH' (sgr-0c05d93507d4b7efc), and 'All ICMP - IPv4' (sgr-08587a4e61824eabe). The 'SSH' rule is highlighted, showing its configuration: Protocol: TCP, Port range: 22, and Source: vpc-0c15a6b46dba96671.

Name	Security group rule ID	IP version	Type	Protocol	Port range
-	sgr-07d2e61acc343314a	-	All traffic	All	All
-	sgr-0c05d93507d4b7efc	IPv4	SSH	TCP	22
-	sgr-08587a4e61824eabe	IPv4	All ICMP - IPv4	ICMP	All

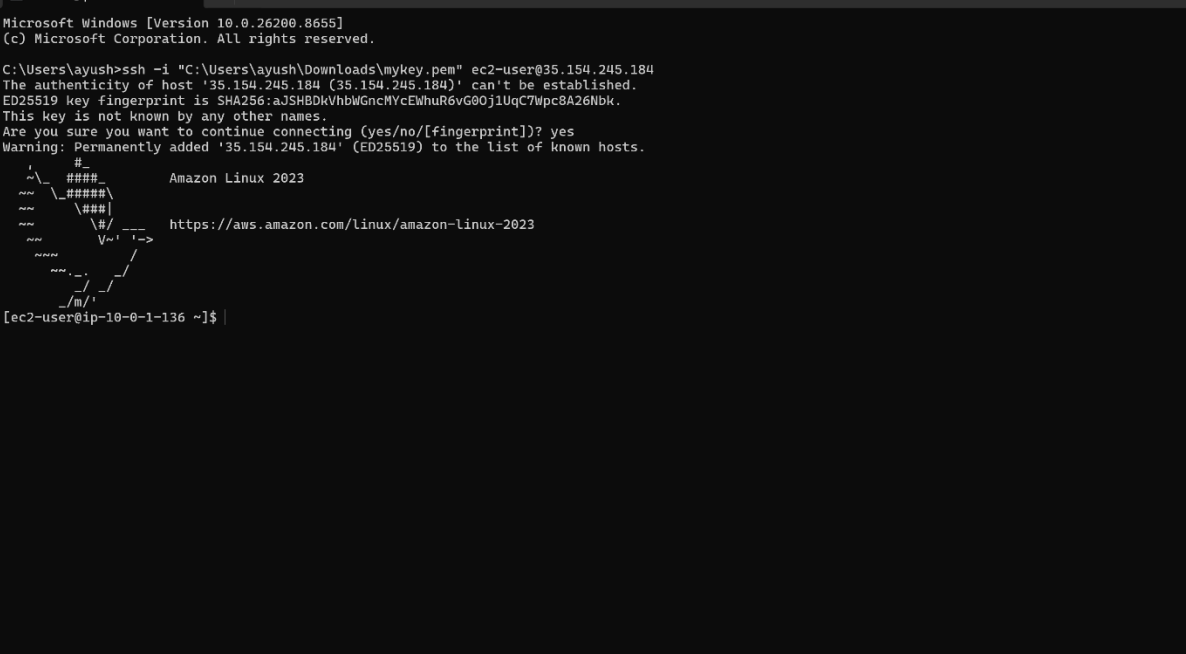
3. Testing, Validation, and Verification

1. Launch an EC2 virtual instance inside the VPC-A-Public subnet, ensuring a public IP address is assigned.
2. Launch a separate EC2 virtual instance inside the VPC-B-Private subnet with no public IP allocation.
3. Connect securely to the public instance in VPC-A via SSH from your local workstation.
4. From the command terminal of the VPC-A instance, execute a network ping directly to the hidden private IP address of the VPC-B private instance:

Bash

ping 10.1.2.237

```
C:\Users\ayush>ssh -i "C:\Users\ayush\Downloads\mykey.pem" ec2-user@35.154.245.184  
Microsoft Windows [Version 10.0.26200.8655]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\ayush>ssh -i "C:\Users\ayush\Downloads\mykey.pem" ec2-user@35.154.245.184  
The authenticity of host '35.154.245.184 (35.154.245.184)' can't be established.  
ED25519 key fingerprint is SHA256:aJSHBDkVhbWGncMYcENhuR6vG0jlUqC7Wpc8A26Nbk.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '35.154.245.184' (ED25519) to the list of known hosts.
```



```
#  
#####          Amazon Linux 2023  
#####  
#####  
#####  
#####  
#####  
#####  
#####  
#####  
#####  
#####  
#####
```

```
https://aws.amazon.com/linux/amazon-linux-2023  
  
[ec2-user@ip-10-0-1-136 ~]$
```

```
ec2-user@ip-10-0-1-136:~$ ping 10.1.2.237
PING 10.1.2.237 (10.1.2.237) 56(84) bytes of data:
64 bytes from 10.1.2.237: icmp_seq=1 ttl=127 time=0.201 ms
64 bytes from 10.1.2.237: icmp_seq=2 ttl=127 time=0.211 ms
64 bytes from 10.1.2.237: icmp_seq=3 ttl=127 time=0.209 ms
64 bytes from 10.1.2.237: icmp_seq=4 ttl=127 time=0.200 ms
64 bytes from 10.1.2.237: icmp_seq=5 ttl=127 time=0.225 ms
64 bytes from 10.1.2.237: icmp_seq=6 ttl=127 time=0.205 ms
64 bytes from 10.1.2.237: icmp_seq=7 ttl=127 time=0.205 ms
64 bytes from 10.1.2.237: icmp_seq=8 ttl=127 time=0.208 ms
64 bytes from 10.1.2.237: icmp_seq=9 ttl=127 time=0.258 ms
64 bytes from 10.1.2.237: icmp_seq=10 ttl=127 time=0.233 ms
64 bytes from 10.1.2.237: icmp_seq=11 ttl=127 time=0.205 ms
64 bytes from 10.1.2.237: icmp_seq=12 ttl=127 time=0.233 ms
64 bytes from 10.1.2.237: icmp_seq=13 ttl=127 time=0.229 ms
^C
--- 10.1.2.237 ping statistics ---
13 packets transmitted, 13 received, 0% packet loss, time 12503ms
rtt min/avg/max/mdev = 0.200/0.217/0.258/0.016 ms
[ec2-user@ip-10-0-1-136 ~]$
```

4.Conclusion

In this project, I successfully connected two separate networks (VPC-A and VPC-B) using **AWS VPC Peering**. By doing this, the servers inside these two different VPCs were able to communicate with each other using their Private IPs.

The successful "**Ping**" test proved that I configured the Route Tables and Security Groups correctly. The best part of this setup is that the data transfer between the VPCs happens on the AWS private network. This makes it highly secure because the data doesn't travel over the public internet.

At the end of the project, I also deleted all the resources (Cleanup process) like EC2 instances, VPCs, and Peering Connections to follow AWS best practices and avoid any extra billing charges.